```
require './controller'
```

When you `require './controller'`, the module loader finds the controller.coffee file relative to the current directory, loads it, and evaluates it. To load a local file, you can also use `../.`, which looks in the parent directory. This works the same as file paths on your command line.

Using `require` is only half the equation, though; blog.coffee still won't compile:

```
# -- blog.coffee –

require './controller'

class Blog extends Controller
```

You still get a reference error:

```
coffee –c blog.coffee
# ReferenceError: Controller is not defined
```

To use the `Controller` class inside the blog file, you first need to `export` it from the controller file.

12.1.2. Exporting

To make a property available to other modules via `require`, you assign it to `exports` inside the module. You do that at the end of the file:

```
# -- controller.coffee –

class Controller

exports.Controller = Controller
```

Now you can `require` the `controller` module and access the `Controller` property it assigned to `exports`:

```
# -- blog.coffee –-

controller = require './controller'
```